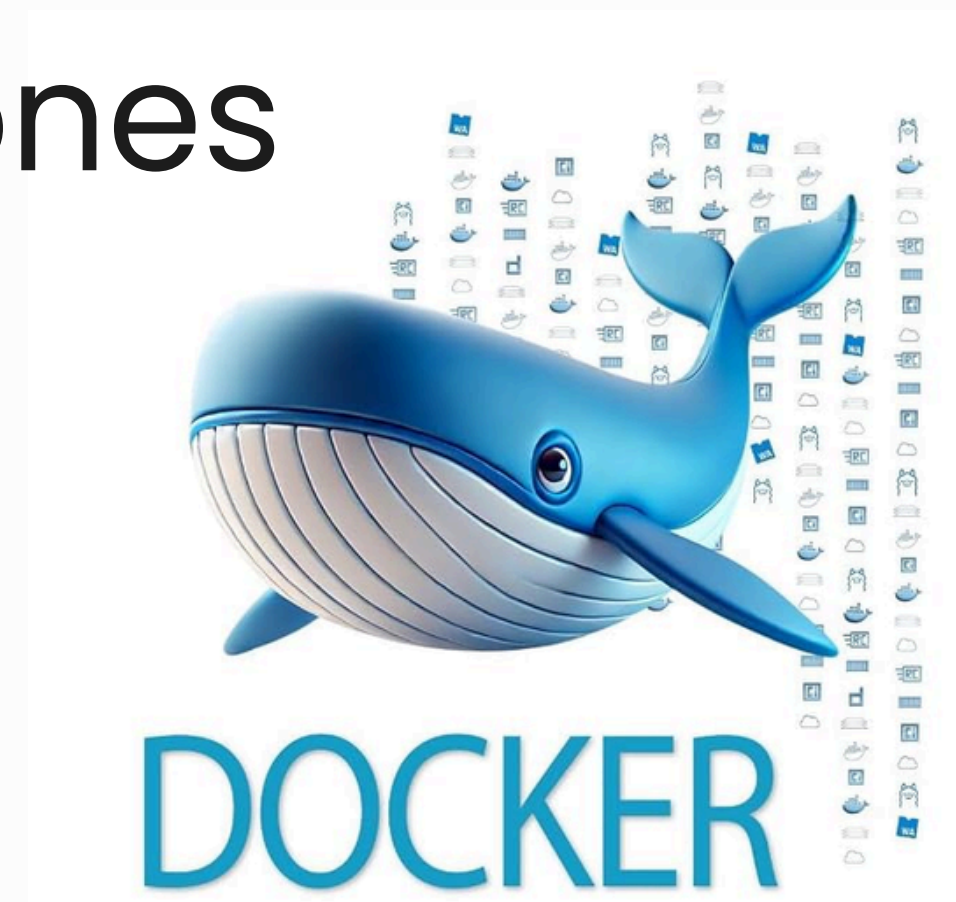


DOCKER DESDE CERO:

Crea y Despliega Aplicaciones

INSTRUCTOR: Cristian Jampier Chileno Segundo
Oficina de Tecnologías de la Información (OTI)
Programa Completo — PIT 2026



SESIÓN 1

- 1 Introduccion y Fundamentos
- 2 Virtualizacion e Hipervisores
- 3 Instalacion de Docker
- 4 Imagenes y Contenedores
- 5 Comandos Esenciales
- 6 Primera Aplicacion

01

Objetivo de la sesion

- Explicar el concepto de contenerizacion.
- Explicar que problema resuelve Docker.
- Ubicar Docker frente a VM e hipervisores.
- Identificar el flujo para instalar una VM base.
- Saber donde instalar Docker y Compose desde la documentacion oficial.
- Diferenciar imagen y contenedor.
- Ejecutar comandos basicos: run, ps, stop, rm.
- Construir y ejecutar una primera app en contenedor.



Menos teoria, mas terminal

Introducción y Fundamentos

EL PROBLEMA: Funciona en mi maquina

- La app funciona en la laptop del desarrollador.
- En otro equipo falla por versiones distintas.
- En produccion aparecen librerias faltantes o configuraciones diferentes.
- El equipo pierde tiempo reconstruyendo el entorno.

CLAVE

Docker busca que el entorno viaje junto con la aplicacion.



“En mi PC funciona”

SOLUCIÓN: La contenerizacion

- Aislar la aplicacion en un contenedor reproducible.
- Empaquetar codigo, dependencias y configuracion en una imagen.
- Compartir esa imagen con el equipo o con un registro central.
- Ejecutarla en distintos entornos con menos diferencias.

Resultado

La app deja de depender de “lo que justo estaba instalado” en una maquina.



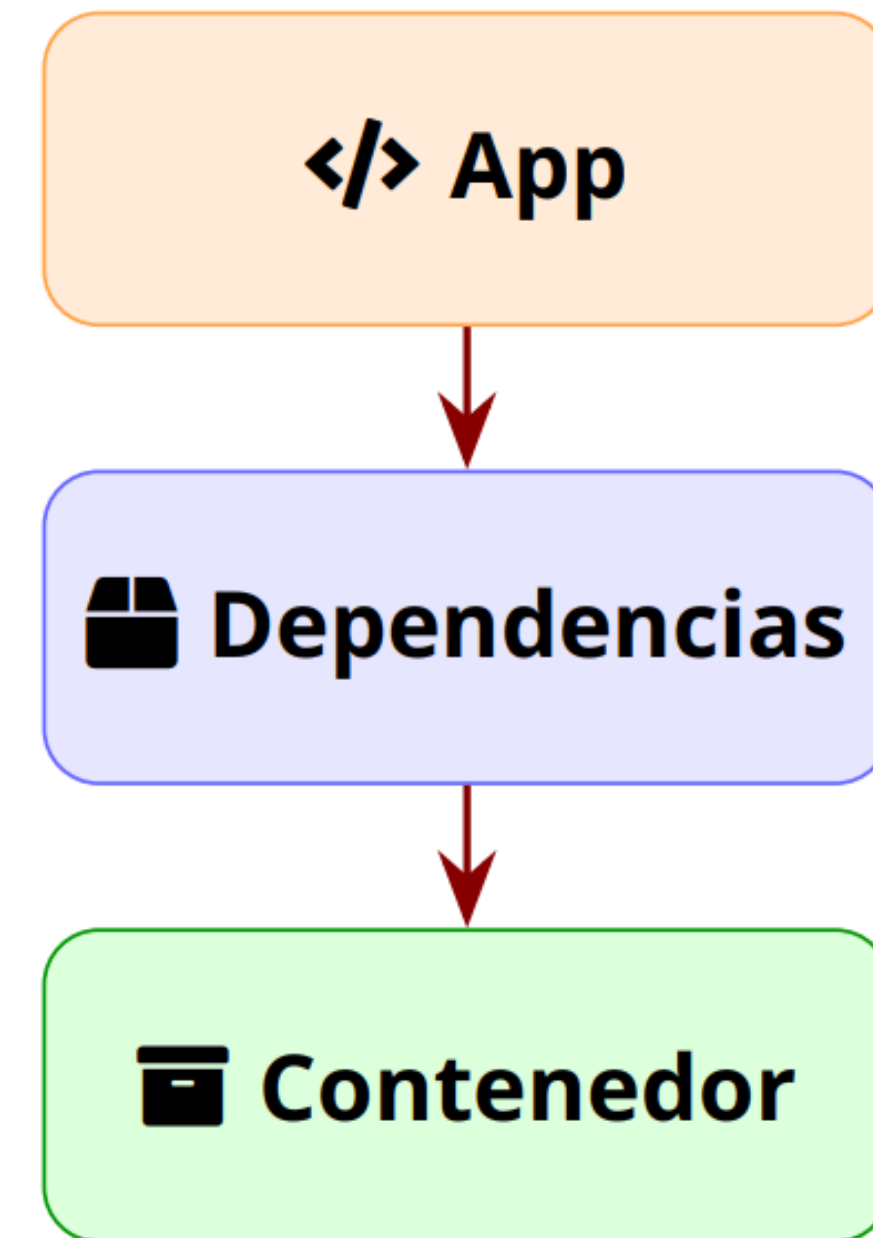
App + dependencias

¿QUE ES LA CONTENERIZACION?

En palabras simples

La contenerizacion empaqueta una aplicacion con lo necesario para ejecutarla de forma uniforme en diferentes entornos.

- Aplicacion + dependencias.
- Configuracion minima reproducible.
- Ejecucion aislada del resto del sistema.
- Menos problemas de compatibilidad.



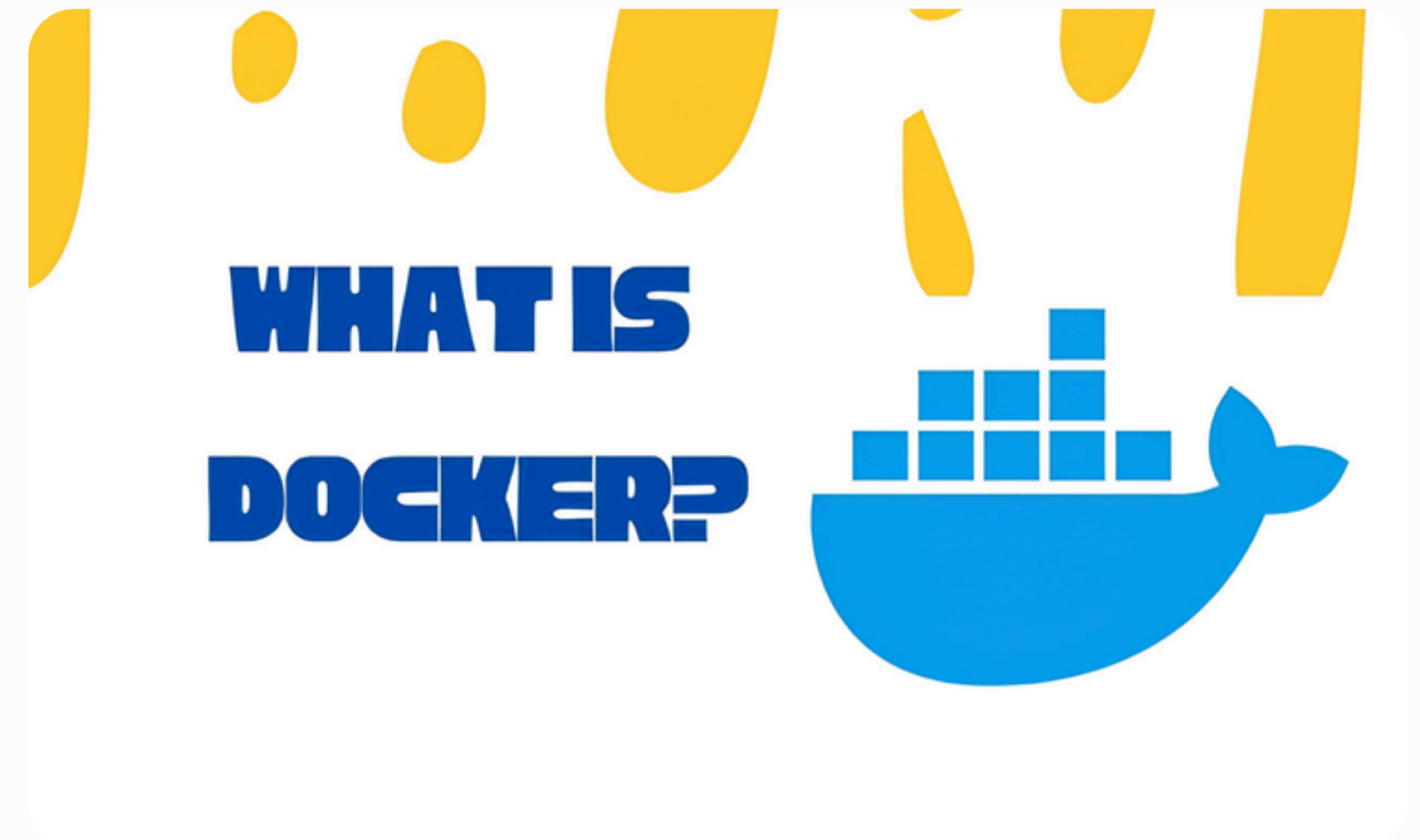
La pregunta natural

Ya tenemos la idea

Contenerizar es empaquetar una app con sus dependencias para ejecutarla igual en distintos entornos.

Ahora falta la herramienta

Docker es una de las formas mas usadas para construir, compartir y ejecutar esos contenedores.



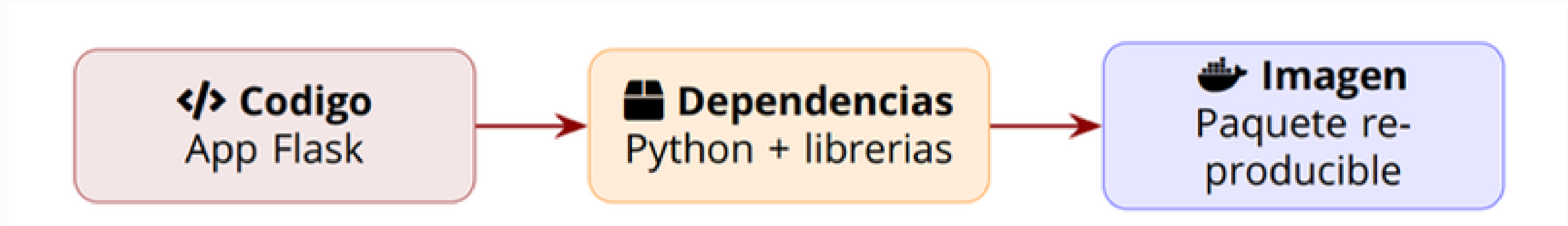
Entonces... que es Docker?

ENTONCES... QUE ES DOCKER

**Si una VM virtualiza
una maquina...**

**Docker ejecuta aplicaciones
en contenedores**

DOCKER EN UNA FRASE



Definición Práctica

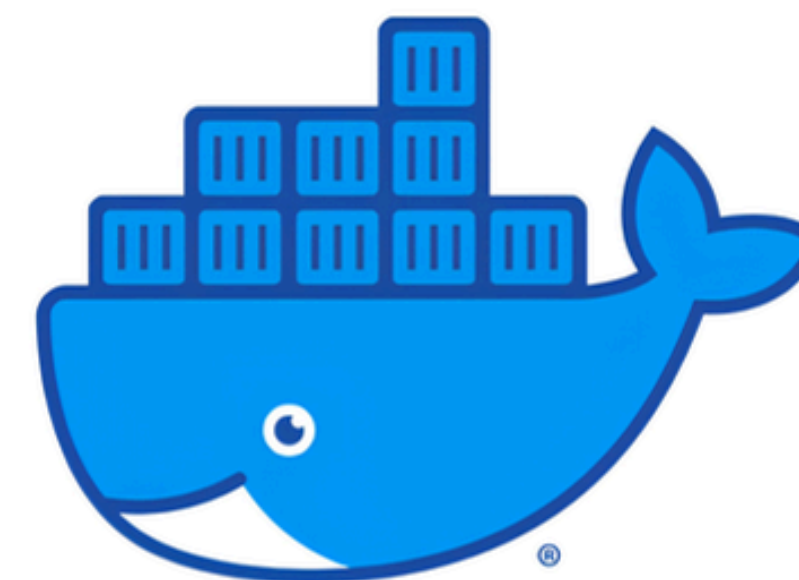
Docker permite empaquetar una aplicación con sus dependencias para ejecutarla de forma consistente en diferentes equipos.

DOCKER: Introducción Práctica

- Docker es una plataforma open source para crear, compartir y ejecutar contenedores.
- Implementa la idea de contenerización en el flujo diario de desarrollo.
- Facilita pruebas, despliegues y trabajo en equipo.
- Se apoya en imágenes, contenedores, registros y la CLI de Docker.

Idea Central

Docker no reemplaza toda VM: resuelve la ejecución reproducible de aplicaciones.



DOCKER

Que si veremos y que no veremos hoy

HOY SI

- Concepto de Docker.
- Concepto de contenerizacion.
- Ruta oficial de instalacion.
- Imagen vs contenedor.
- Comandos esenciales.
- Primera app con Dockerfile.

HOY NO

- Kubernetes, OCI profundo o cgroups.
- YAML avanzado.
- Redes y volumenes a detalle.
- Produccion con Nginx.

PERO... COMO FUNCIONA?

Entendido... ¿Pero como funciona?
¿Es Docker simplemente otra...

Maquina Virtual?

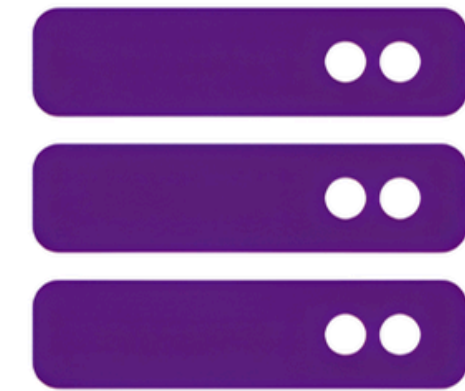
Virtualización e Hipervisores

Antes de Docker: maquinas virtuales

- Una VM ejecuta un sistema operativo completo sobre hardware virtual.
- Permite aislar entornos: Linux, Windows Server, laboratorios, pruebas.
- Es util cuando necesitas simular una maquina completa.
- Consume mas recursos que un contenedor porque incluye su propio kernel.

Idea clave

La VM virtualiza una maquina; Docker empaqueta procesos aislados.



Maquina completa

QUE ES UN HIPERVISOR

Tipo 1

Corre directo sobre el hardware. Ejemplos: VMware ESXi, Hyper-V Server, Proxmox.

Tipo 2

Corre como aplicación del sistema host. Ejemplos: VirtualBox, VMware Workstation.

Para este curso

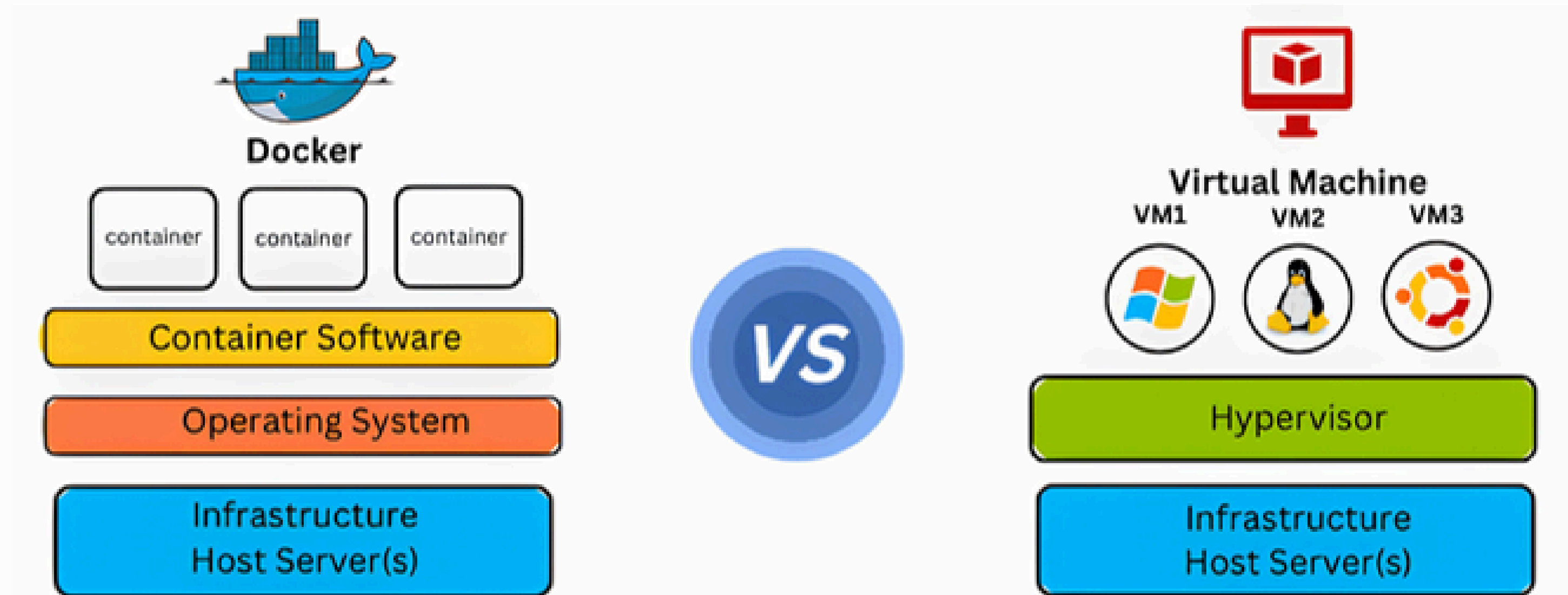
Si necesitas un laboratorio local, una VM con Linux puede servir como entorno limpio para instalar Docker y practicar sin afectar tu sistema principal.

LA SIGUIENTE DUDA

¿Y Docker?

**¿Es un
hipervisor? ¿De
que tipo?**

DOCKER VS MAQUINA VIRTUAL



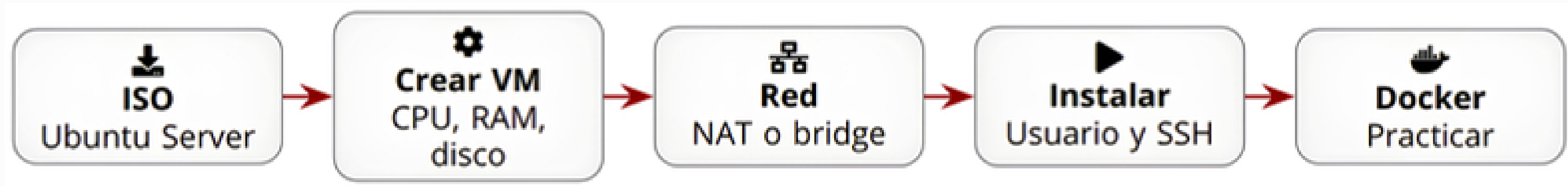
Diferencia clave

La VM incluye un sistema operativo completo (Guest OS). El contenedor comparte el kernel del host y solo incluye lo que la app necesita.

VM VS CONTENEDOR: DECISION RAPIDA

Criterio	Maquina virtual	Contenedor
Sistema operativo	Incluye SO completo	Comparte kernel del host
Arranque	Mas lento	Rapido
Consumo	Mayor RAM y disco	Menor consumo
Aislamiento	Fuerte a nivel maquina	Aislado a nivel proceso
Uso comun	Servidores y laboratorios	Apps reproducibles

INSTALANDO UNA VM: FLUJO RECOMENDADO



Configuracion sugerida

2 CPU, 4 GB RAM, 25 GB disco, red NAT para empezar.
Si necesitas acceso desde otros equipos, usa bridge.

Instalacion de Docker

QUE DEBES INSTALAR



Criterio del curso

No memorices comandos de instalacion: pueden cambiar por sistema operativo.
Usa siempre la guia oficial y valida segun tu plataforma.

Ruta oficial segun tu sistema

Windows y macOS

- Instalar Docker Desktop.
- Incluye Docker Engine, CLI y Docker Compose.
- Revisar requisitos de WSL 2 en Windows.

 <https://docs.docker.com/desktop/>

Linux o VM Linux

- Instalar Docker Engine segun distribucion.
- Revisar pasos post-instalacion.
- Instalar Compose desde la guia oficial.

 <https://docs.docker.com/engine/install/>

DOCKER COMPOSE Y ENLACES OFICIALES

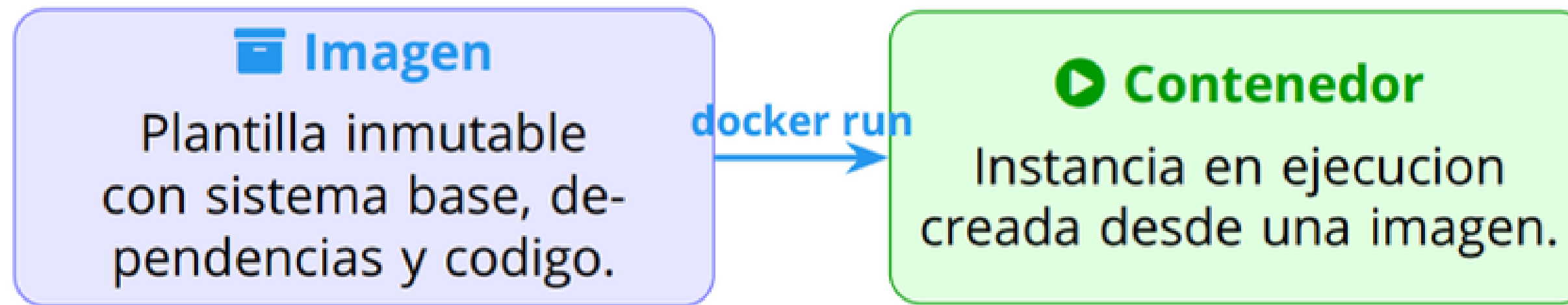
TEMA	CUANDO USARLO	DOCUMENTACIÓN
Docker Desktop	Windows/MacOS o Experiencia integrada	Docker Desktop
Docker Engine Post-instalación Linux	Linux nativo o VM Linux Permisos, servicio y ajustes posteriores	Docker Engine Post-instalación Linux
Docker Compose	Aplicaciones con varios servicios	Docker Compose

Regla practica

Si una guia externa contradice la documentacion oficial, se prioriza Docker Docs.

Imagenes y contenedores

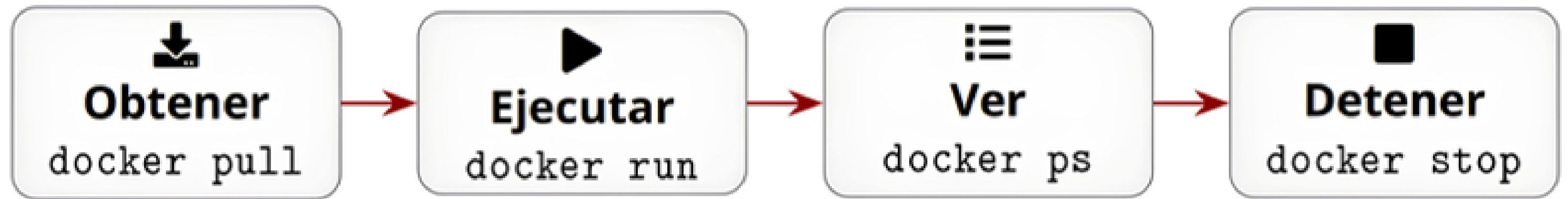
IMAGEN VS CONTENEDOR



Regla mental

La imagen es la receta; el contenedor es el plato servido y ejecutándose.

CICLO DE VIDA BASICO



Lo minimo para operar

Ejecutar, listar, detener, eliminar contenedores y revisar imagenes locales.

Comandos Esenciales

LABORATORIO 1: COMPROBAR DOCKER

```
docker --version  
docker info  
docker run hello-world
```

Que validamos

- La CLI responde.
- El Engine esta activo.
- Docker puede descargar y ejecutar imagenes.

LABORATORIO 2: EJECUTAR NGINX

```
docker run --name web-demo -d -p 8080:80 nginx  
docker ps
```

Abrir en navegador:

<http://localhost:8080>

Que significa

- name: nombre del contenedor.
- -d: ejecuta en segundo plano.
- -p 8080:80: publica el puerto.
- nginx: imagen usada.

LABORATORIO 3: DETENER Y LIMPIAR

```
docker stop web-demo  
docker ps -a  
docker rm web-demo  
docker images
```

Buenas practicas

- No acumules contenedores detenidos.
- Usa nombres claros en laboratorios.
- Verifica antes de borrar.

MAPA RAPIDO DE COMANDOS

COMANDO	CUANDO USARLO	DOCUMENTACIÓN
<code>docker run</code>	Crea y ejecuta un contenedor	<code>docker run nginx</code>
<code>docker ps</code>	Lista contenedores activos	<code>docker ps -</code>
<code>docker stop</code>	Detiene un contenedor	<code>docker stop web-</code>
<code>docker rm</code>	Elimina contenedores detenidos	<code>demo</code> <code>docker rm web-</code>
<code>docker images</code>	Lista imagenes locales	<code>demo</code> <code>docker images</code>

Primera aplicación

PROYECTO PRACTICO DE HOY



META

Crear una app minima, construir su imagen y verla funcionando desde el navegador.

PASO 1: CREAR ARCHIVOS DEL PROYECTO

```
cd codigos/sesion1  
  
# Archivos ya preparados para la practica:  
# app.py  
# requirements.txt  
# Dockerfile
```

Estructura esperada

Los archivos de la app estan en codigos/sesion1 junto al material de esta sesion.

PASO 2: CÓDIGO DE LA APP FLASK

```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route("/")
6  def home():
7      return "Hola Docker desde PIT 2026"
8
9  if __name__ == "__main__":
10     app.run(host="0.0.0.0", port=5000)
```

PASO 3: DEPENDENCIAS

```
# requirements.txt  
flask==3.0.3
```

Por qué declararlas

El contenedor debe poder instalar lo necesario sin depender de lo que tenga la laptop del alumno.

PASO 4: DOCKERFILE MÍNIMO

```
1 FROM python:3.12-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7
8 COPY app.py .
9
10 EXPOSE 5000
11 CMD ["python", "app.py"]
```


PASO 5: CONSTRUIR LA IMAGEN

```
docker build -t mi-flask:v1 .  
docker images
```

Lectura del comando

-t mi-flask:v1 asigna nombre y versión a la imagen. El punto final indica que el contexto de build es la carpeta actual.

PASO 6: EJECUTAR LA APLICACIÓN

```
docker run --name flask-demo -d -p 5000:5000 mi-flask:v1  
docker ps  
# Abrir:  
# http://localhost:5000
```

Resultado esperado

El navegador debe mostrar: Hola Docker desde PIT 2026

PASO 7: LIMPIAR EL LABORATORIO

```
docker stop flask-demo  
docker rm flask-demo
```

```
# Opcional: borrar la imagen  
docker rmi mi-flask:v1
```

Resultado esperado

Primero detienes el contenedor, luego lo eliminas. La imagen queda disponible hasta que decidas borrarla.

MAPA RAPIDO DE COMANDOS

ERROR	CAUSA PROBABLE	SOLUCIÓN RÁPIDA
Puerto ocupado	Otro proceso usa 5000	Cambiar a -p 5001:5000
No abre en navegador	Contenedor detenido	Revisar docker ps -a
Imagen no encontrada	Nombre mal escrito	Revisar docker images
Build falla	Archivo faltante o typo	Verificar carpeta y Dockerfile

CHECKLIST DE APRENDIZAJE — SESIÓN 1

- Puedo explicar qué es un hipervisor y para qué sirve una VM.
- Puedo describir el flujo básico para instalar una VM de laboratorio.
- Puedo ubicar la documentación oficial para instalar Docker y Compose.
- Puedo explicar qué es la contenerización y qué problema resuelve.
- Puedo explicar por qué Docker evita diferencias entre entornos.
- Puedo diferenciar imagen y contenedor con un ejemplo.
- Puedo ejecutar, listar, detener y eliminar contenedores.
- Puedo construir una imagen propia con docker build.
- Puedo ejecutar mi primera app usando docker run -p.